Fixed-Point Quantization

How the Lane Centering Assist (LCA) Tiny CNN turns trained decimals into whole numbers, and how the FPGA computes with them without ever seeing a decimal point.

> **The numbers here are examples.** The real `Fw1`, `Fa`, `Fw2` and `S1` come from training, in `mem/quant_params.txt`, which doesn't exist yet. This page uses `Fw1 = 7`, `Fa = 6`, `Fw2 = 7` and made-up weights. The formulas are the ones in the README (number formats).

1. Why we need it | 2. What it means | 3. The two rules | 4. Add, subtract, multiply, divide | 5. Fw1, Fa, Fv

---

1. Why we need it

Training gives decimals like `-0.62` and `0.05`. You could run those on the FPGA as floating point, but each floating-point adder or multiplier costs hundreds of LUTs and takes several cycles. It's also harder to match Python exactly.

Integers suit the hardware much better. One DSP48 does an integer multiply-add every cycle, and the project spec only requires fixed-point RTL anyway.

The trade-off is a small rounding error. Check G2 limits the accuracy drop from float to int8 to 2 percentage points.

---

2. What it means

Fixed-point means computing with whole numbers while everyone agrees where the decimal point sits. Think of money stored in cents: `1234` means $12.34. The computer only stores 1234, and the point exists only by agreement.

We do the same with a power of two instead of 100. `F` is the number of fractional bits:

```
real value = stored integer / 2^F
```

With `F = 7`, a stored `79` means 79/128 = 0.6172. Inside the 8 bits, the agreed point sits here:

`0` . `1` `0` `0` `1` `1` `1` `1`

Quantization is the reverse step, done once in Python:

```
stored = round(real × 2^F), clamped to the type's range

-0.62 × 128 = -79.36  →  -79
```

With int8 and F = 7	Value
Smallest real value	`-128/128 = -1.0`
Largest real value	`127/128 = 0.992`
Step size	`1/128 = 0.0078`
Worst rounding error	`half a step = 0.0039`

Choosing F is a range vs precision trade-off. A bigger F gives finer steps but a smaller range.

---

3. The two rules everything follows

Rule 1: multiplying adds the fractional bits

```
(x / 2^8) × (w / 2^7) = (x × w) / 2^15
```

The hardware just computes `x × w`, and the result's scale is now 2^{-15}.

Rule 2: you can only add numbers on the same scale

It's like adding cents to cents. That's why the bias is stored at the accumulator's scale, not the weights' scale.

> **Shifting right by S changes F to F − S** and costs almost nothing in hardware. This is why all our scales are powers of two: a rescale is just a shift, with no extra multiplier.

---

4. Add, subtract, multiply and divide

The hardware treats these numbers as plain integers. **Add, subtract, multiply and divide all use the normal integer circuits, and the only extra work is tracking `F`.** It's a fixed point, not a floating one: `F` never changes while the circuit runs, so the hardware doesn't need to store it.

Operation	Integer calculation	Fractional bits of the result	Extra step
Add	`a + b`	Same `F`	Scales must match first
Subtract	`a - b`	Same `F`	Scales must match first
Multiply	`a × b`	`Fa + Fb`	Usually shift back down
Divide	`a / b`	`Fa - Fb`	Shift `a` up first

The examples in this section use `F = 4`, where stored = real × 16. So `1.5` is stored as `24`, and `0.25` is stored as `4`. In this table, `Fa` and `Fb` mean the fractional bits of `a` and `b`, not the project's activation scale.

Add and subtract

With the same `F`, you just use the integers directly:

```
1.5 + 0.25  →  24 + 4 = 28   →  28/16 = 1.75   ✓
1.5 - 0.25  →  24 - 4 = 20   →  20/16 = 1.25   ✓
0.25 - 1.5  →  4 - 24 = -20  →  -20/16 = -1.25 ✓
```

If the scales differ, line them up first. Take `1.5` with `F = 4` (stored `24`) and `0.25` with `F = 2` (stored `1`). Adding `24 + 1` would be wrong, because 24 means sixteenths and 1 means quarters. So shift the `F = 2` number left by 2 first:

```
1 << 2 = 4
24 + 4 = 28  →  28/16 = 1.75 ✓
```

This is why our bias is stored at the accumulator's scale. An add can also need one more bit than its inputs.

Multiply

You don't need to line anything up. Multiply the integers and add the `F` values:

```
1.5 × 0.25  →  24 × 4 = 96, with F = 4 + 4 = 8
96 / 256 = 0.375 ✓
```

The result has more fractional bits and needs more storage, because 8 bits × 8 bits gives 16 bits. To get back to `F = 4`, drop 4 bits with rounding:

```
(96 + 8) >>> 4 = 104 / 16 = 6.5 → 6
6 / 16 = 0.375 ✓
```

That's the same step as our `S1` requantize. Dropping bits is where precision can be lost, though this example happened to come out exact.

Divide

Integer division subtracts the `F` values, so the fractional bits can disappear:

```
1.5 / 0.25  →  24 / 4 = 6,  with F = 4 - 4 = 0  →  6 ✓
0.25 / 1.5  →  4 / 24 = 0,  with F = 0          →  0 ✗ (should be 0.1667)
```

The fix is to shift the top number up before dividing:

```
(4 << 4) / 24 = 64 / 24 = 2.67 → 2, with F = 4
2 / 16 = 0.125  (real answer 0.1667)
```

The result is close, but limited by `F`. A larger `F` would give a closer answer.

A true divider is also slow and large in hardware, usually taking many cycles, so designs avoid it in two ways:

- **Dividing by a power of two:** this is just a right shift, and the circuit is only wiring.
- **Dividing by a fixed number:** multiply by its reciprocal instead. For `x / 3`, use `(x × 85) >>> 8`, because 85/256 ≈ 0.332. It's a little inexact: for `x = 120` it gives 39 instead of 40.

What a right shift really does

A shift can mean two different things, depending on what you decide about `F`:

- **Keep `F` the same:** the real value is divided. `24 >>> 1 = 12`, and with `F = 4` that's 0.75, half of 1.5.
- **Lower `F` by the shift amount:** the real value stays the same. `24 >>> 1 = 12`, and with `F = 3` that's still 1.5.

Our `S1` shift is the second kind. It changes the scale from `8 + Fw1` to `Fa` and keeps the real value, apart from rounding.

Where each operation is used in our CNN

Operation	Where
Multiply	Both MACs: `pixel × weight` and `activation × weight`
Add	Accumulators: products and bias
Subtract	Not in the datapath, because negative weights just give negative products
Divide	Never a real divider, only the `>>> S1` shift
Compare	Max pooling and argmax, which work directly because the values share one scale

---

5. What Fw1, Fa, Fw2 and S1 are

The first three say how many fractional bits a number has. `S1` is a shift amount calculated from the others.

Name	Stands for	What it sets	Example	Real value
`Fw1`	Fractional bits of weights, layer 1	Scale of the conv weights	7	`w / 128`
`Fa`	Fractional bits of activations	Scale after ReLU, also used by pooling and FC	6	`a / 64`
`Fw2`	Fractional bits of weights, layer 2	Scale of the FC weights	7	`w / 128`
`S1`	Shift, layer 1	Right shift after convolution	9	Not a scale

- **Fw1 and Fw2:** Python picks the largest value where the biggest weight in that layer still fits in int8. If the biggest is 0.62, 7 works (`0.62 × 128 = 79`) but 8 doesn't (`0.62 × 256 = 159`). The layers are chosen separately, so they can differ.
- **Fa:** chosen from measured activations on real images. A larger value gives finer steps, but more values get stuck at 127. The maximum real value is `127 / 2^Fa`, which is 1.98 for `Fa = 6`. Check G4 requires under 1% of values to clamp.
- **S1:** not chosen freely. The accumulator has `8 + Fw1` fractional bits and we want `Fa`, so we drop the difference.

```
S1 = 8 + Fw1 - Fa = 8 + 7 - 6 = 9
```

`Fw1`, `Fw2` and `Fa` never appear in the RTL. They only affect the integers Python exports to the `.mem` files. `S1` is the only one the RTL uses directly, in `relu_requant`. There's no `S2` because the logits are never rescaled.

---

6. The scales in our network

Pixel	× Conv weight	Accumulator	>>> S1, clamp	Max pool

uint8
F = 8

int8
F = Fw1

24 bit
F = 8 + Fw1

uint7
F = Fa

uint7
F = Fa

× FC weight
int8
F = Fw2

Logits
int32
F = Fa + Fw2

Value	Type	Real value	Example scale
Pixel `x`	uint8	x / 2^8	1/256
Conv weight	int8	w / 2^Fw1	1/128
Conv accumulator and bias	24 / 32 bit	acc / 2^(8+Fw1)	1/32768
Activation `a`	uint7	a / 2^Fa	1/64
Pool output	uint7	same as `a`	1/64
FC weight	int8	w / 2^Fw2	1/128
FC accumulator, bias, logits	int32	logit / 2^(Fa+Fw2)	1/8192

7. One full calculation

Step 1: quantize (done once in Python)

Weights × 128, rounded:

```
float                    int8
-0.31   0.02   0.33       -40    3    42
-0.62   0.05   0.60   →   -79    6    77
-0.29  -0.01   0.31       -37   -1    40
```

The bias is `0.10`, stored at the accumulator's scale: `0.10 × 32768 = 3277`.

Step 2: convolution

The 3×3 window of pixels is dark on the left and bright on the right:

```
40  120  200
50  130  210
60  140  220
```

The accumulator starts at the bias, then adds one product per cycle:

Tap	x × w	Product	Accumulator
start	bias		3,277
0	40 × -40	-1,600	1,677
1	120 × 3	360	2,037
2	200 × 42	8,400	10,437
3	50 × -79	-3,950	6,487
4	130 × 6	780	7,267
5	210 × 77	16,170	23,437
6	60 × -37	-2,220	21,217
7	140 × -1	-140	21,077
8	**220 × 40**	**8,800**	**29,877**

> **How close is it?** `29877 / 32768 = 0.9118`, and the same sum in floats gives `0.9082`. The error is only 0.0036, and the hardware never used a decimal.

Step 3: ReLU and requantize

The accumulator has 15 fractional bits and we want 6, so `S1 = 9` and `round = 1 << (S1-1) = 256`:

```
a = clamp((acc + round) >>> S1, 0, 127)

(29877 + 256) >>> 9 = 30133 / 512 = 58.85 → 58
```

So `a = 58`, which means `58/64 = 0.906`. Here's what each part does:

- `+ round` **rounds to nearest.** A shift alone always rounds down. With `acc = 1000`, `1000 >>> 9` gives 1, even though 1000/512 is 1.95. Adding 256 first gives 2.
- `>>>` **keeps the sign.** It's an arithmetic shift. It rounds toward minus infinity, which matches `>>` on signed integers in Python.
- **The clamp is the ReLU.** The lower limit of 0 removes negatives, and the upper limit of 127 is saturation.

Case	acc	After shift	After clamp
Negative	-5,000	-10	0 (ReLU)
Normal	29,877	58	58
Too big	70,000	137	127 (saturation)

Step 4: max pooling

This is only comparisons, with no arithmetic. For the 2×2 group `58, 12, 3, 41`, the output is `58`. The scale doesn't change.

Step 5: fully connected

FC weights have `Fw2 = 7`, so `-0.18` becomes `-23`. The product scale is `6 + 7 = 13` bits, or 1/8192. A bias of `0.25` becomes `0.25 × 8192 = 2048`. Here's a shortened version with 3 inputs instead of 420:

```
acc = 2048
acc += 58 × -23 = -1334  →  714
acc += 12 ×  40 =   480  →  1194
acc +=  0 ×  90 =     0  →  1194
```

So `logit = 1194`, which means `1194 / 8192 = 0.146`.

Step 6: argmax

There's no shift here, because all three logits share one scale and argmax only compares them. With invented logits:

```
LEFT = 1194, STRAIGHT = 3010, RIGHT = -420  →  class 1, STRAIGHT
```

---

8. Bit widths

Zero-extending the pixel

The DSP multiply is signed, so an unsigned pixel needs a leading 0. Pixel 200 is `11001000`, which reads as **-56** if treated as signed. Then `200 × 42` wrongly becomes `-2,352`. With `{1'b0, x}` it's a 9-bit positive 200, and the product is correctly `8,400`.

Why activations are uint7

Their top bit is always 0 in the 8-bit memory, so FC can read them as signed without any extension. The FC product therefore stays 16 bits.

Accumulator overflow

Overflow is a silent bug: the value wraps around and becomes wrong, so we size for the worst case.

Accumulator	Largest product	Terms	Worst sum	Needed (with sign)	Used
Conv	255 × 128 = 32,640	9	293,760	20 bits, plus the bias	24
FC	127 × 128 = 16,256	420	6,827,520	24 bits, plus the bias	32

The docs say 25 bits for FC because they use the safe general rule: 16 bits + `ceil(log2(420))` = 25.

---

9. Why the accumulator is int24 and the bias is int32

The docs don't give a reason, so this is a reading of the design: **the two widths were chosen for different reasons, and in practice the bias value fits in 24 bits anyway.** The 32 is the storage format, and the 24 is what the calculation needs.

- **Bias at 32 bits:** decision D5 uses one format for both layers' biases, and the FC accumulator really is 32 bits. All biases sit in `.mem` files as 8 hex digits.
- **Accumulator at 24 bits:** the conv sum needs only 20 bits plus room for the bias. Going to 32 would add 8 bits to the adder and registers without changing any result.

Cutting 32 bits to 24 is safe because a two's-complement number keeps its value when you drop top bits, as long as those bits are only copies of the sign:

```
 3277 = 00000000 00000000 00001100 11001101
-3277 = 11111111 11111111 11110011 00110011
        └─ dropped ┘  (8 bits, all copies of the sign)
```

How big the bias can be: the accumulator's limit is ±8,388,607, and the products can reach ±293,760. That leaves about ±8,090,000 for the bias, or a real bias of about ±247 with `Fw1 = 7`. A trained bias is normally far smaller, around ±1.

> **The risk:** if a bias ever exceeded that, its top bits would be cut off with no error. The fix is a check in the export script, plus a deliberate truncation in the RTL:

```
# Python, for each conv bias
assert abs(bias) + 293760 < 2**23

// Verilog
wire signed [23:0] bias24 = bias_rom_data[23:0];
```

---

10. In Verilog

```
wire signed [8:0]  x9   = {1'b0, pixel};          // zero-extend
wire signed [16:0] prod = x9 * weight;            // 9b × 8b = 17b
acc <= acc + prod;                                // starts at the bias
```

```
wire signed [23:0] rounded = acc + (1 << (S1-1));
wire signed [23:0] shifted = rounded >>> S1;
assign a = (shifted < 0)   ? 8'd0   :
           (shifted > 127) ? 8'd127 : shifted[7:0];
```

> **The scales never appear in the hardware.** They only show up indirectly, in the `S1` shift and in the integers from Python. The RTL doesn't know that 58 means 0.906. That's why Python's integer model is the specification: it uses these same integer steps, with no floats, so every stage can match the RTL bit for bit.

---

11. Try it yourself

Quantize a value

Real value: -0,62

F (fractional bits): 7

```
-0.62 × 2^7 = -79.36
stored int8 = -79
real value  = -0.6172
error       = 0.0028
```

ReLU and requantize

Accumulator: 29877

Fw1: 7

Fa: 6

```
S1      = 8 + 7 - 6 = 9
round   = 256
shifted = (29877 + 256) >>> 9 = 58
a       = 58
real    = 0.9063
```

Sources: `README.md` (number formats), `DESIGN_DECISIONS.md` D5 and D25.